Task 3: Project Management Tool

Express + SQLite + JWT + WebSockets (ws). Group projects, members, task board (To do/Doing/Done), assignments, task comments, live notifications (bonus).

GitHub repository name: **CodeAlpha_ProjectManagementTool**

How to run

```
cd CodeAlpha_ProjectManagementTool
npm install
npm start        # open http://localhost:3000  (Node 18+)
```

Overview

Features: auth, group projects, add members, task board with To do/Doing/Done, assignments, task comments. Bonus: live notifications and board refresh over WebSockets. Tables: users, projects, members, tasks, comments.

Emoji in the UI are omitted from this listing only (the PDF font cannot show them).

CodeAlpha_ProjectManagementTool/package.json

```
{ "name": "codealpha-projects", "version": "1.0.0", "main": "server.js", "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.2", "better-sqlite3": "*", "bcryptjs": "^2.4.3", "jsonwebtoken": "^9.0.2",
      "ws": "^8.18.0" } }
```

CodeAlpha_ProjectManagementTool/auth.js

```
const jwt = require('jsonwebtoken'), bcrypt = require('bcryptjs');
const SECRET = process.env.JWT_SECRET || 'dev-secret-change-me';
const sign = u => jwt.sign({ id: u.id, username: u.username }, SECRET, { expiresIn: '7d' });
const verify = t => jwt.verify(t, SECRET);
const auth = (req, res, next) => {
  try { req.user = verify((req.headers.authorization || '').replace('Bearer ', '')); next(); }
  catch { res.status(401).json({ error: 'Login required' }); }
};
// Registers POST /api/register and /api/login (needs a `users(id, username, password)` table)
function authRoutes(app, db) {
  app.post('/api/register', (req, res) => {
    const { username, password } = req.body || {};
    if (!/^\w{3,20}$/.test(username || '') || (password || '').length < 6)
      return res.status(400).json({ error: 'Username 3-20 chars (letters/digits/_), password 6+ chars' });
    try {
      const id = db.prepare('INSERT INTO users(username,password) VALUES(?,?)').run(username,
          bcrypt.hashSync(password, 10)).lastInsertRowid;
      res.json({ token: sign({ id, username }), username });
    } catch { res.status(409).json({ error: 'Username already taken' }); }
  });
  app.post('/api/login', (req, res) => {
    const { username, password } = req.body || {};
    const u = db.prepare('SELECT * FROM users WHERE username=?').get(username || '');
    if (!u || !bcrypt.compareSync(password || '', u.password)) return res.status(401).json({ error: 'Wrong username
        or password' });
    res.json({ token: sign(u), username: u.username });
  });
}
module.exports = { sign, verify, auth, authRoutes };
```

CodeAlpha_ProjectManagementTool/server.js

```
const express = require('express'), http = require('http'), { WebSocketServer } = require('ws'), Database =
    require('better-sqlite3');
const { auth, authRoutes, verify } = require('./auth');
const db = new Database('pm.db');
db.exec(`
CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, username TEXT UNIQUE, password TEXT);
CREATE TABLE IF NOT EXISTS projects(id INTEGER PRIMARY KEY, name TEXT, owner_id INTEGER);
CREATE TABLE IF NOT EXISTS members(project_id INTEGER, user_id INTEGER, PRIMARY KEY(project_id,user_id));
CREATE TABLE IF NOT EXISTS tasks(id INTEGER PRIMARY KEY, project_id INTEGER, title TEXT, status TEXT DEFAULT 'todo',
    assignee_id INTEGER, created_by INTEGER);
CREATE TABLE IF NOT EXISTS comments(id INTEGER PRIMARY KEY, task_id INTEGER, user_id INTEGER, body TEXT, created TEXT
    DEFAULT CURRENT_TIMESTAMP);`);
const app = express(); app.use(express.json()); app.use(express.static('public')); authRoutes(app, db);
const server = http.createServer(app), wss = new WebSocketServer({ server }), clients = new Set();

// WebSockets: real-time notifications + live board refresh
wss.on('connection', (s, req) => {
  try { s.uid = verify(new URL(req.url, 'http://x').searchParams.get('token')).id; clients.add(s); s.on('close', ()
      => clients.delete(s)); }
  catch { s.close(); }
```

```
});
const notify = (pid, text, except) => {
  const ids = db.prepare('SELECT user_id FROM members WHERE project_id=?').all(pid).map(r => r.user_id);
  clients.forEach(c => { if (ids.includes(c.uid) && c.uid !== except && c.readyState === 1) c.send(JSON.stringify({
      projectId: pid, text })); });
};
const isMember = (pid, uid) => !!db.prepare('SELECT 1 FROM members WHERE project_id=? AND user_id=?').get(pid, uid);
const taskFor = (req, res) => {
  const t = db.prepare('SELECT * FROM tasks WHERE id=?').get(req.params.id);
  if (!t || !isMember(t.project_id, req.user.id)) { res.status(403).json({ error: 'No access to this task' }); return
      null; }
  return t;
};

app.get('/api/projects', auth, (req, res) => res.json(db.prepare('SELECT p.* FROM projects p JOIN members m ON
    m.project_id=p.id WHERE m.user_id=? ORDER BY p.id DESC').all(req.user.id)));
app.post('/api/projects', auth, (req, res) => {
  const name = (req.body.name || '').trim(); if (!name) return res.status(400).json({ error: 'Name required' });
  const id = db.prepare('INSERT INTO projects(name,owner_id) VALUES(?,?)').run(name, req.user.id).lastInsertRowid;
  db.prepare('INSERT INTO members VALUES(?,?)').run(id, req.user.id); res.json({ id });
});
app.get('/api/projects/:id', auth, (req, res) => {
  const pid = +req.params.id; if (!isMember(pid, req.user.id)) return res.status(403).json({ error: 'Not a member' });
  res.json({
    project: db.prepare('SELECT * FROM projects WHERE id=?').get(pid),
    members: db.prepare('SELECT u.id,u.username FROM members m JOIN users u ON u.id=m.user_id WHERE
        project_id=?').all(pid),
    tasks: db.prepare('SELECT t.*, u.username assignee, (SELECT COUNT(*) FROM comments WHERE task_id=t.id) comments
        FROM tasks t LEFT JOIN users u ON u.id=t.assignee_id WHERE project_id=? ORDER BY t.id').all(pid)
  });
});
app.post('/api/projects/:id/members', auth, (req, res) => {
  const pid = +req.params.id; if (!isMember(pid, req.user.id)) return res.status(403).json({ error: 'Not a member' });
  const u = db.prepare('SELECT id FROM users WHERE username=?').get(req.body.username || '');
  if (!u) return res.status(404).json({ error: 'No such user' });
  db.prepare('INSERT OR IGNORE INTO members VALUES(?,?)').run(pid, u.id);
  notify(pid, `${req.user.username} added ${req.body.username} to the project`, req.user.id); res.json({ ok: true });
});
app.post('/api/projects/:id/tasks', auth, (req, res) => {
  const pid = +req.params.id, title = (req.body.title || '').trim();
  if (!isMember(pid, req.user.id)) return res.status(403).json({ error: 'Not a member' });
  if (!title) return res.status(400).json({ error: 'Title required' });
  const a = req.body.assignee_id || null; if (a && !isMember(pid, a)) return res.status(400).json({ error: 'Assignee
      must be a member' });
  const id = db.prepare('INSERT INTO tasks(project_id,title,assignee_id,created_by) VALUES(?,?,?,?)').run(pid, title,
      a, req.user.id).lastInsertRowid;
  notify(pid, `${req.user.username} created task "${title}"`, req.user.id); res.json({ id });
});
app.put('/api/tasks/:id', auth, (req, res) => {
  const t = taskFor(req, res); if (!t) return;
  const { status, assignee_id } = req.body;
  if (status !== undefined) {
    if (!['todo', 'doing', 'done'].includes(status)) return res.status(400).json({ error: 'Bad status' });
    db.prepare('UPDATE tasks SET status=? WHERE id=?').run(status, t.id);
    notify(t.project_id, `${req.user.username} moved "${t.title}" to ${status}`, req.user.id);
  }
  if (assignee_id !== undefined) {
    if (assignee_id && !isMember(t.project_id, assignee_id)) return res.status(400).json({ error: 'Assignee must be a
        member' });
    db.prepare('UPDATE tasks SET assignee_id=? WHERE id=?').run(assignee_id || null, t.id);
    notify(t.project_id, `${req.user.username} reassigned "${t.title}"`, req.user.id);
  }
  res.json({ ok: true });
});
app.get('/api/tasks/:id/comments', auth, (req, res) => {
  const t = taskFor(req, res); if (!t) return;
  res.json(db.prepare('SELECT c.body,c.created,u.username FROM comments c JOIN users u ON u.id=c.user_id WHERE
      task_id=? ORDER BY c.id').all(t.id));
});
app.post('/api/tasks/:id/comments', auth, (req, res) => {
  const t = taskFor(req, res); if (!t) return;
  const body = (req.body.body || '').trim(); if (!body) return res.status(400).json({ error: 'Empty comment' });
  db.prepare('INSERT INTO comments(task_id,user_id,body) VALUES(?,?,?)').run(t.id, req.user.id, body);
  notify(t.project_id, `${req.user.username} commented on "${t.title}"`, req.user.id); res.json({ ok: true });
});
server.listen(process.env.PORT || 3000, () => console.log('PM tool on http://localhost:3000'));
```

CodeAlpha_ProjectManagementTool/public/index.html

```
<!doctype html><html><head><meta charset="utf-8"><meta name="viewport"
    content="width=device-width,initial-scale=1"><title>TaskAlpha</title><link rel="stylesheet"
    href="style.css"></head>
<body><header><b> TaskAlpha</b><span id="nav"></span></header><main id="app"></main><div id="toast"></div>
<script src="common.js"></script>
<script>
let pid = null, tid = null, ws, members = [];
const COLS = [['todo', ' To do'], ['doing', ' In progress'], ['done', ' Done']];
function nav() { $('#nav').innerHTML = token ? `<a onclick="projects()">Projects</a><a onclick="logout()">Logout
```

```
    (${esc(me)})</a>` : ''; }
function toast(t) { const d = document.createElement('div'); d.textContent = ' ' + t; $('#toast').append(d);
    setTimeout(() => d.remove(), 3500); }
function connect() {
  if (!token || ws) return;
  ws = new WebSocket((location.protocol === 'https:' ? 'wss://' : 'ws://') + location.host + '/?token=' + token);
  ws.onmessage = e => { const m = JSON.parse(e.data); toast(m.text); if (pid === m.projectId) board(pid, true); };
  ws.onclose = () => { ws = null; setTimeout(connect, 2000); };
}
async function projects() {
  pid = null; const ps = await api('/projects');
  $('#app').innerHTML = `<div class="card"><input id="pn" placeholder="New project name"><button
      onclick="newProject()">Create project</button></div>` +
    ps.map(p => `<div class="card" style="cursor:pointer"
        onclick="board(${p.id})"><b>${esc(p.name)}</b></div>`).join('');
}
async function newProject() { try { await api('/projects', 'POST', { name: $('#pn').value }); projects(); } catch (e)
    { alert(e.message); } }
async function board(id, quiet) {
  pid = id; const d = await api('/projects/' + id); members = d.members;
  const opts = sel => `<option value="">Unassigned</option>` + members.map(m => `<option value="${m.id}" ${m.id ==
      sel ? 'selected' : ''}>${esc(m.username)}</option>`).join('');
  const side = $('#side') ? $('#side').innerHTML : '';
  $('#app').innerHTML = `<h2>${esc(d.project.name)}</h2>
  <div class="card"><b>Members:</b> ${members.map(m => esc(m.username)).join(', ')}<br><input id="mu"
      placeholder="Add member by username" style="width:60%"><button onclick="addMember()">Add</button></div>
  <div class="card"><input id="tt" placeholder="New task title" style="width:45%"><select id="ta"
      style="width:30%">${opts()}</select><button onclick="addTask()">Add task</button></div>
  <div class="cols">` + COLS.map(([s, label]) => `<div><h3>${label}</h3>` + d.tasks.filter(t => t.status === s).map(t
      => `<div class="card"><b>${esc(t.title)}</b><br>
    <select onchange="assign(${t.id},this.value)">${opts(t.assignee_id)}</select>
    ${COLS.filter(c => c[0] !== s).map(c => `<button class="alt" onclick="move(${t.id},'${c[0]}')">->
        ${c[0]}</button>`).join(' ')}
    <button onclick="openTask(${t.id})"> ${t.comments}</button></div>`).join('') + '</div>').join('') + `</div><div
        id="side"></div>`;
  if (tid && quiet) openTask(tid);
}
const act = f => async (...a) => { try { await f(...a); await board(pid); } catch (e) { alert(e.message); } };
const addMember = act(() => api(`/projects/${pid}/members`, 'POST', { username: $('#mu').value }));
const addTask = act(() => api(`/projects/${pid}/tasks`, 'POST', { title: $('#tt').value, assignee_id: +$('#ta').value
    || null }));
const move = act((id, status) => api('/tasks/' + id, 'PUT', { status }));
const assign = act((id, v) => api('/tasks/' + id, 'PUT', { assignee_id: +v || null }));
async function openTask(id) {
  tid = id; const cs = await api(`/tasks/${id}/comments`);
  $('#side').innerHTML = `<div class="card"><h3>Task discussion</h3>${cs.map(c => `<div
      class="cm"><b>${esc(c.username)}</b>: ${esc(c.body)}</div>`).join('') || '<i>No comments yet</i>'}
  <input id="cb" placeholder="Write a comment, press Enter" onkeydown="if(event.key==='Enter')comment()"></div>`;
}
async function comment() { try { await api(`/tasks/${tid}/comments`, 'POST', { body: $('#cb').value }); await
    board(pid); openTask(tid); } catch (e) { alert(e.message); } }
nav(); if (token) { connect(); projects(); } else $('#app').innerHTML = authBox();
</script></body></html>
```

CodeAlpha_ProjectManagementTool/public/common.js

```
const $ = s => document.querySelector(s);
let token = localStorage.token || '', me = localStorage.me || '';
async function api(path, method = 'GET', body) {
  const r = await fetch('/api' + path, { method, headers: { 'Content-Type': 'application/json', Authorization:
      'Bearer ' + token }, body: body ? JSON.stringify(body) : undefined });
  const d = await r.json().catch(() => ({}));
  if (r.status === 401 && token) { localStorage.clear(); location.reload(); }
  if (!r.ok) throw new Error(d.error || 'Something went wrong');
  return d;
}
async function authSubmit(kind) {
  try { const d = await api('/' + kind, 'POST', { username: $('#u').value, password: $('#p').value });
    token = localStorage.token = d.token; me = localStorage.me = d.username; location.reload();
  } catch (e) { alert(e.message); }
}
function logout() { localStorage.removeItem('token'); localStorage.removeItem('me'); location.reload(); }
const esc = s => String(s ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;',
    "'": '&#39;' }[c]));
const authBox = () => `<div class="card narrow"><h3>Login / Register</h3><input id="u" placeholder="username"><input
    id="p" type="password" placeholder="password (6+ chars)"><button onclick="authSubmit('login')">Login</button>
    <button class="alt" onclick="authSubmit('register')">Register</button></div>`;
```

CodeAlpha_ProjectManagementTool/public/style.css

```
*{box-sizing:border-box}body{margin:0;font-family:system-ui,sans-serif;background:#f4f5f7;color:#222}
header{display:flex;justify-content:space-between;align-items:center;padding:12px
    18px;background:#1f2937;color:#fff;position:sticky;top:0;z-index:5}
header a{color:#cbd5e1;margin-left:14px;text-decoration:none;cursor:pointer}header a:hover{color:#fff}
main{max-width:1000px;margin:18px auto;padding:0 12px}
.card{background:#fff;border-radius:10px;padding:14px;margin-bottom:12px;box-shadow:0 1px 3px
    #0002}.narrow{max-width:360px;margin:40px auto}
input,textarea,select{width:100%;padding:9px;margin:5px 0;border:1px solid #cbd5e1;border-radius:6px;font:inherit}
```

```
button,.btn{background:#2563eb;color:#fff;border:0;padding:8px
    14px;border-radius:6px;cursor:pointer;font:inherit;display:inline-block}
button.alt,.btn.alt{background:#64748b}button:hover{opacity:.9}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(210px,1fr));gap:12px}.big{font-size:56px;text-align:c
    enter}
.cols{display:grid;grid-template-columns:repeat(3,1fr);gap:12px}@media(max-width:700px){.cols{grid-template-columns:1f
    r}}
.cm{background:#f1f5f9;border-radius:6px;padding:6px 9px;margin:5px 0;font-size:14px}
#toast div{position:fixed;right:14px;bottom:14px;background:#111;color:#fff;padding:10px 14px;border-radius:8px}
```